

Assignment 8:

Step 1: Problem Statement

Create a series of data structures to hold information read from 3 different text files. Then, using iterators to look through the structures, we will create a "decoding" machine which will allow us to leverage a keygrid to decode associated encrypted messages.

This assignment will provide practice on array lists, 2D arrays, queues, and iterators. It will require us to read from files, populate arrays, arraylists, etc. and, through the application of a variety of classes & methods manipulate the values read from the text file to demonstrate a rudimentary method for decrypting text files.

Step 2: What I know

- pretty good with the classes needed and their functions
- pretty good with arraylists, 2D arrays, queues, and the like.
- Reading from files is no issue

Step 2: What I don't know

- I haven't worked with iterators before. I have a concept of their intent and syntax, so I'm hoping this will only be a small learning curve.

Step 3: Pseudocode

```
// Create File obj for reading
```

```
File myFile = new File("File1.Txt");  
" " "2 = " " (" "2.Txt");  
" " "3 = " " (" "3.txt");
```

```
// create scanners to read
```

```
Scanner myScanner = new Scanner(myFile);  
" " "2 = " " ("myFile2);  
" " "3 = " " ("myFile3);
```

```
// Create decoding Machine Obj
```

```
decodingMachine myDecoder = new dec.Mech();
```

```
// create array list
```

```
ArrayListCharmyFile myList = new ArrayList<>();
```

```
Scanner.nextInt = Row; Scanner.nextInt = Column
```

```
Loop to fill list
```

```
ArrayList<i> = scanner.next;
```

```
// loop
```

```
// Create iterator
```

```
iterator<character> myIt = new iterator<>();
```


Call fillKeyGrid (char iterator);

// Create queues to hold messages

Queue<String> myQ1 = new LinkedList<>();

Queue<Int> myQ2 = new LinkedList<>();

Iterator<Int> Q1Int = new --- - - - ;
// <int> Q2Int = new --- - - - ;

Call fillQueue // fill Q1

Call fillQueue // fill Q2

loop Call Print KeyGrid;

Print: Call decode message (Iterator1)
// loop

loop print call decod mess (Iterator2)

// loop

// end main

Decoding Machine

private Key Grid = Array[][];

private int numRows = rows;

private int numcols = columns;

// ~~Constructor~~ constructor

public DecodingMachine(int numRows, int numcols)

This.numRows = numRows;

this.numcols = numcols;

Array<int>[] my Array = new Array[numRows][numcols]

Step 3: (cont)

// No Getters or Setters

// Methods

```
public void fillKeyGrid fillKeyGrid(Iterator<Character>  
    charIterator)
```

~~while charIt.hasNext()~~

~~for (i = 0; i < numRows; i++)~~

```
for (i = 0; i < numRows; i++) {  
    if charIt.hasNext?
```

```
    for (j = 0; j < numCols; j++) {  
        if charIt.hasNext?
```

```
        myArray[i][j] = charIt.next;
```

```
    }
```

```
public Iterator<Character> decodeMessage(Iterator  
    <KeyGridElement> msgIterator)
```

```
    ArrayList<Character> myList = secret Message new ArrayList<>
```

```
    for each i in msgIterator
```

```
        secretMessage(i) = msgIterator.next
```

```
    // end for
```

```
    new iterator<Character> snIt = new Iterator
```

```
    return snIt
```

```
public void printKeyGrid()
```

```
    loop
```

```
        print my KD Array
```

```
    end loop
```

```
    // end class
```


Step 4: what I learned

Using iterators is a little tricky But really useful. One mistake I made was using the next() method too often when populating my 2D Array

```
Ex | Character ch = keyGrid[msgIterator.next().getRow()]  
      [msgIterator.next().getColowm()]
```

It took me a minute to realize why that didn't work and to correct my code

Additionally, it took me a minute to successfully use the return from the decode message method for the same reason.

Other than that, just a matter of working through it all step by step. I loved the secret message. A fun project.